k-Nearest Neighbors

Tự tìm láng giềng và tính một dự đoán phân loại hoặc hồi quy.

Một hồ sơ mới

Điểm thuật toán và giờ tự học.

Muốn tham khảo những hồ sơ gần giống.

Đổi giờ thành phút có nên đổi nhóm học?

CÂU HỎI

“Gần” được định nghĩa bằng gì, và thông tin từ hàng xóm được gộp thế nào?

01

Láng giềng và khoảng cách

Ba bước kNN

1. Tính khoảng cách từ truy vấn đến các mẫu train.
2. Chọn k mẫu gần nhất.
3. Phân loại: bỏ phiếu; hồi quy: trung bình.

X: `(n,d)` · q: `(d,)` · khoảng cách: `(n,)`

Không học vector trọng số; phải giữ dữ liệu tham chiếu để dự đoán.

HỎI NHANH

Có ba láng giềng: một láng giềng nhãn 0 ở khoảng cách 1, hai láng giềng nhãn 1 ở khoảng cách 4 và 5. Bỏ phiếu đều thì ra nhãn nào? Bỏ phiếu theo trọng số $1/d$ thì ra nhãn nào?

A Bỏ phiếu đều ra 0, trọng số $1/d$ ra 1

B Bỏ phiếu đều ra 1, trọng số $1/d$ ra 0

C Cả hai cách đều ra nhãn 1

D Cả hai cách đều ra nhãn 0

Chọn hình học

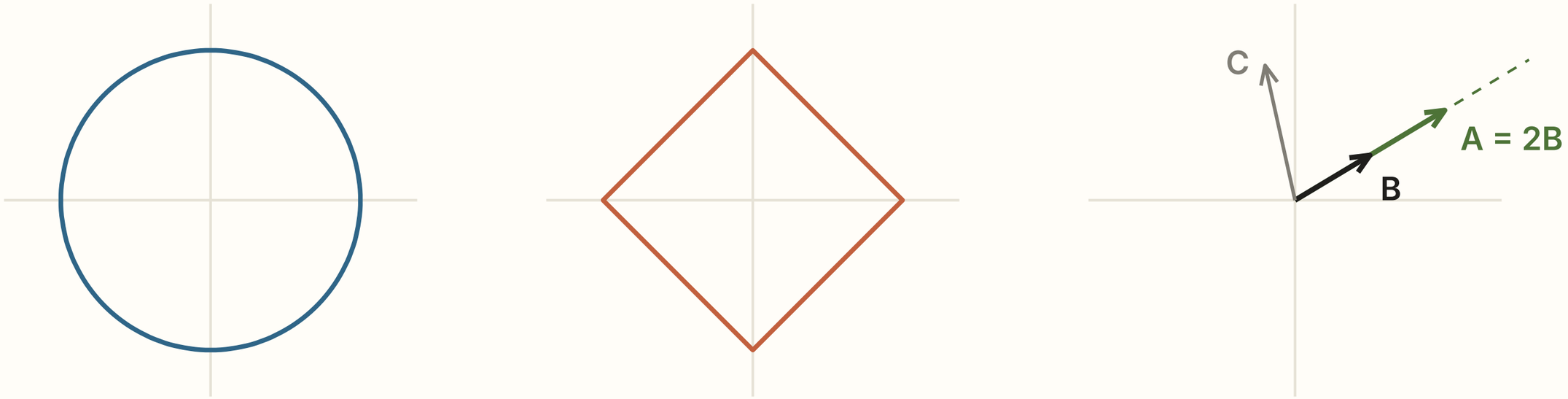
Euclid bình phương:

$$d^2(\mathbf{q}, \mathbf{x}) = \sum_j (q_j - x_j)^2.$$

Manhattan: $\sum_j |q_j - x_j|$; cosine: $1 - \frac{\mathbf{q}^\top \mathbf{x}}{\|\mathbf{q}\|\|\mathbf{x}\|}$.

Với hai vector đơn vị, $\|\mathbf{q} - \mathbf{x}\|^2 = 2 - 2\cos(\mathbf{q}, \mathbf{x})$. Cosine chưa xác định cho vector toàn 0.

Ba cách đo “gần”



Euclid

Manhattan

Cosine

Euclid và Manhattan: các điểm cách gốc đúng 1. Cosine chỉ xét hướng: A = 2B có khoảng cách 0 tới B, còn C khác hướng thì xa.

HỎI NHANH

Văn bản A là văn bản B chép lại hai lần, nên vector đếm từ của A đúng bằng 2 lần vector của B. Theo cosine và theo Euclid, A và B có gần nhau không?

A Cả hai khoảng cách đều bằng 0

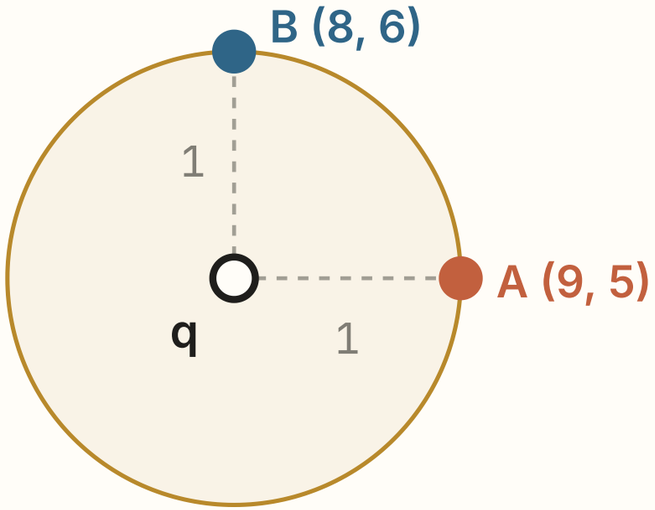
B Khoảng cách cosine lớn, Euclid bằng 0

C Cả hai khoảng cách đều lớn

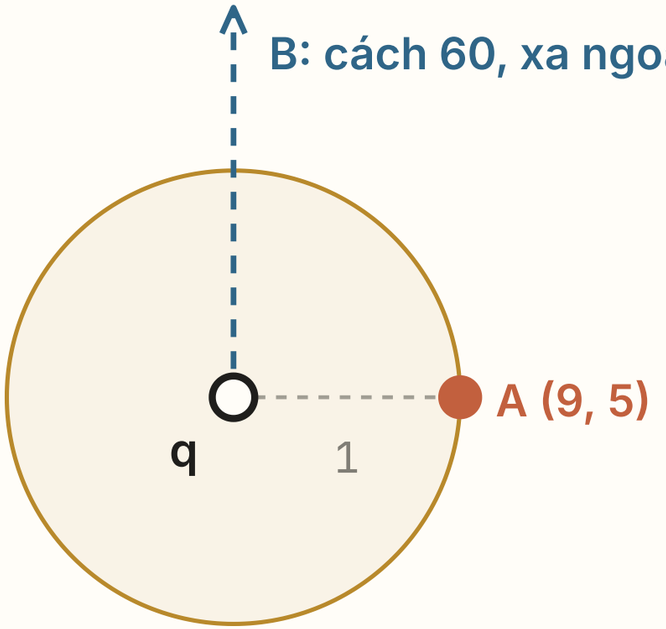
D Khoảng cách cosine bằng 0, Euclid khá lớn

Thang đo là một phần của mô hình

giờ tự học tính bằng giờ



giờ tự học tính bằng phút



Cùng ba hồ sơ, chỉ đổi giờ sang phút: B từ cách q một đơn vị thành cách 60, nên A thành láng giềng duy nhất.

Tính khoảng cách khi đổi giờ sang phút

Truy vấn `(8 điểm, 5 giờ)`, A=`(9,5)`, B=`(8,6)`.

Theo đơn vị hiện tại: $d_A^2 = d_B^2 = 1$.

Đổi giờ sang phút: $d_A^2 = 1$, còn $d_B^2 = 60^2 = 3600$.

StandardScaler học trung bình và độ lệch chuẩn trên train. Nếu muốn một cột có trọng số lớn hơn, đó là giả định cần kiểm bằng validation.

HỎI NHANH

Mỗi ảnh là một embedding 512 chiều, và ta tìm ảnh giống nhau bằng cosine. Muốn chạy kNN theo Euclid cho nhanh, ta chuẩn hóa thế nào?

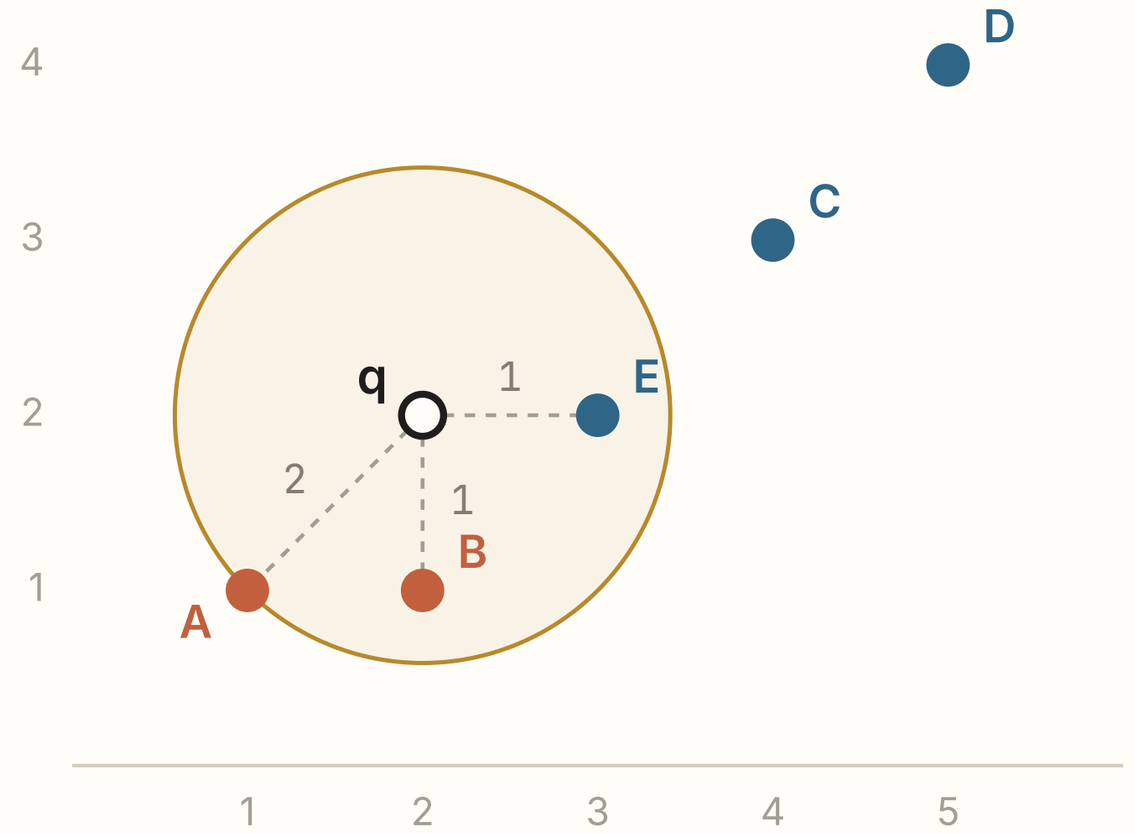
A Dùng `StandardScaler` theo từng cột của bảng embedding

B Chia mỗi hàng cho độ dài (norm) của chính nó

C Đưa từng cột về đoạn $[0, 1]$ theo min và max

D Không cần, vì Euclid và cosine luôn xếp hạng như nhau

Ba láng giềng của q=(2,2)



đỏ: A, B

xanh: C, D, E

k = 3: B, E, A

→ đỏ thắng 2–1

Truy vấn q = (2, 2), số trên nét đứt là d^2: ba láng giềng gần nhất B, E, A; đỏ thắng 2–1.

Tính trên giấy

q=(2,2), k=3

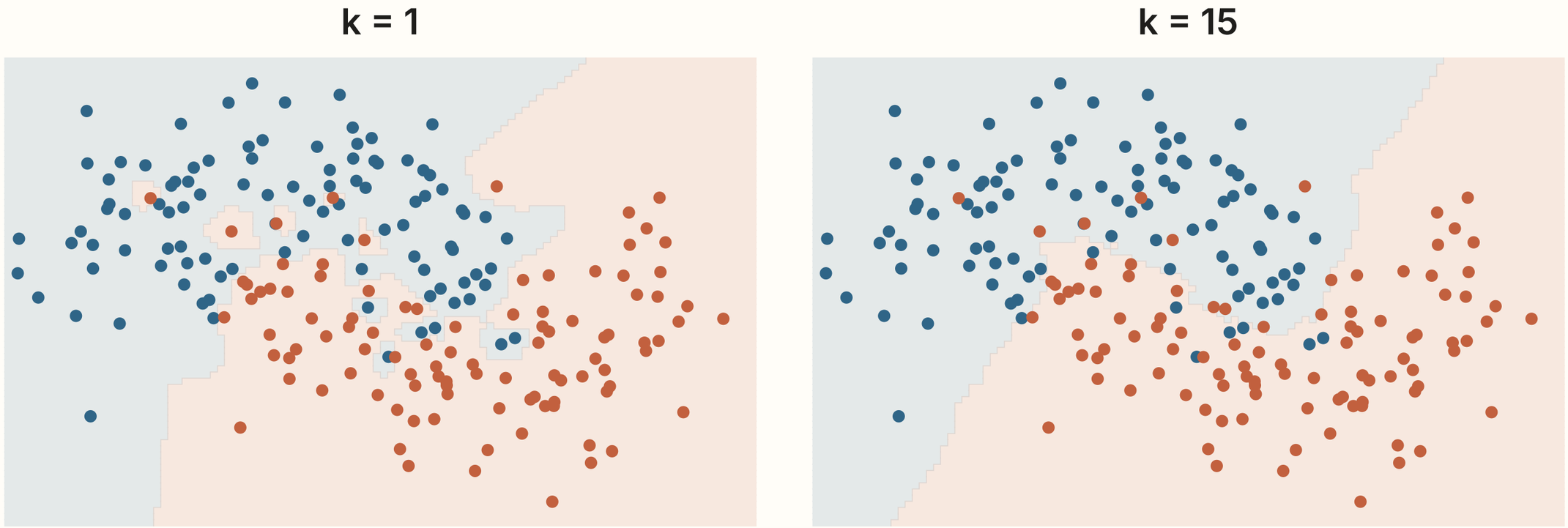
Điểm	A	B	C	D	E
d^2	2	1	5	13	1
Nhãn	đỏ	đỏ	xanh	xanh	xanh

B,E,A là ba láng giềng.

Hai đỏ, một xanh → đỏ.

k=1: B/E hòa khoảng cách, cần quy tắc riêng.

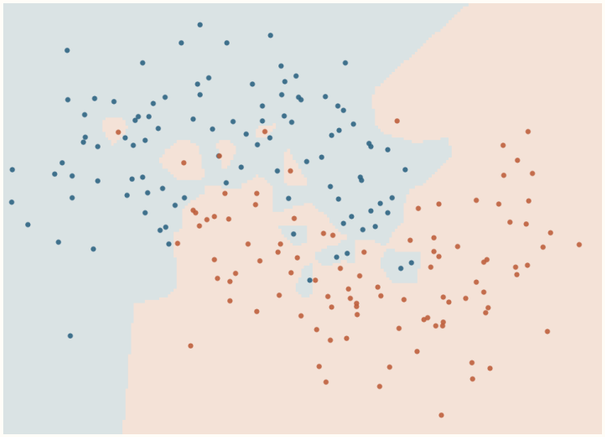
k điều khiển độ mượt



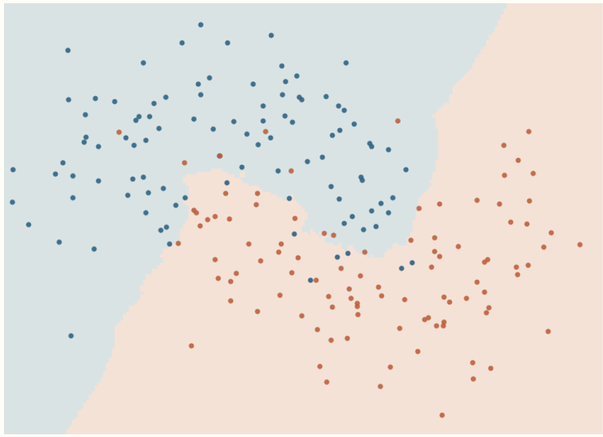
Cùng 200 điểm: k = 1 vẽ ranh giới bám từng điểm nhiễu; k = 15 cho ranh giới mượt.

$k = 1$ bám sát từng mẫu và thường có variance cao. k lớn làm ranh giới mượt hơn; với phiếu đều, $k = n$ chọn lớp đông nhất trong phân loại.

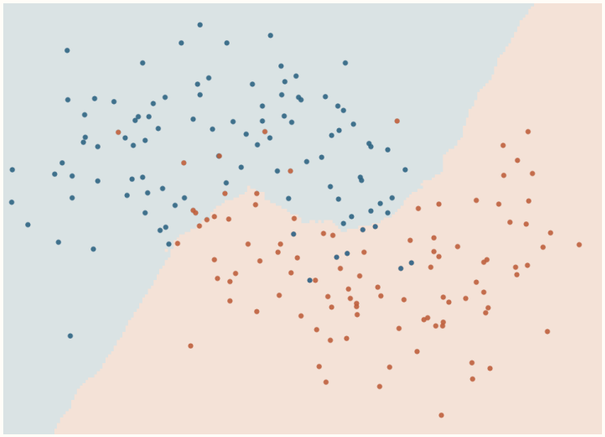
Tăng dần k



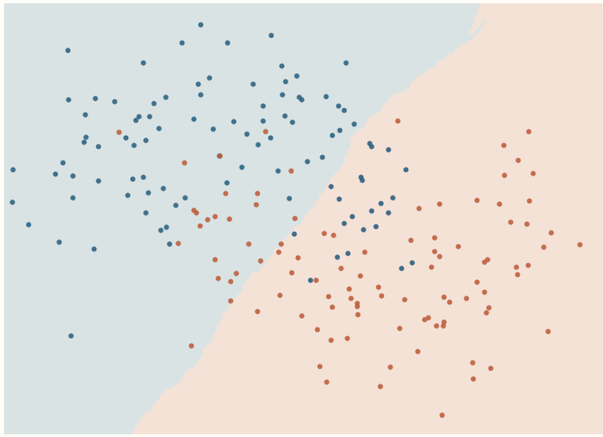
k = 1: accuracy 5-fold CV 0.880



k = 15: accuracy 5-fold CV 0.905



k = 50: accuracy 5-fold CV 0.875



k = 120: accuracy 5-fold CV 0.750

Cùng 200 điểm của hình trên, tăng k từ 1 tới 120: ranh giới đi từ bám từng điểm nhiễu tới quá phẳng.

Đọc điểm train của 1-NN

Mỗi mẫu train là hàng xóm của chính nó, có khoảng cách 0. Nếu không có hồ sơ trùng nhưng nhãn mâu thuẫn, train accuracy thường bằng 1.

Hồ sơ trùng nhau, nhãn khác nhau làm tie cần quy tắc riêng. Khớp train vẫn chưa chứng minh dự đoán mẫu mới tốt.

HỎI NHANH

Một bạn chọn $k = 1$, vì trong các k đã thử, $k = 1$ có lỗi train thấp nhất, bằng 0. Chọn như vậy có ổn không?

A Ổn, vì lỗi train bằng 0 là tốt nhất có thể

B Ổn, nếu trong train không có điểm nào trùng nhau

C Không ổn, phải chọn k lớn nhất đã thử cho an toàn

D Không ổn, vì 1-NN luôn khớp train do mỗi điểm tự là láng giềng của nó

PHỎNG THEO CMU 10-601 · Exam 1 Practice · Q2.3(f)

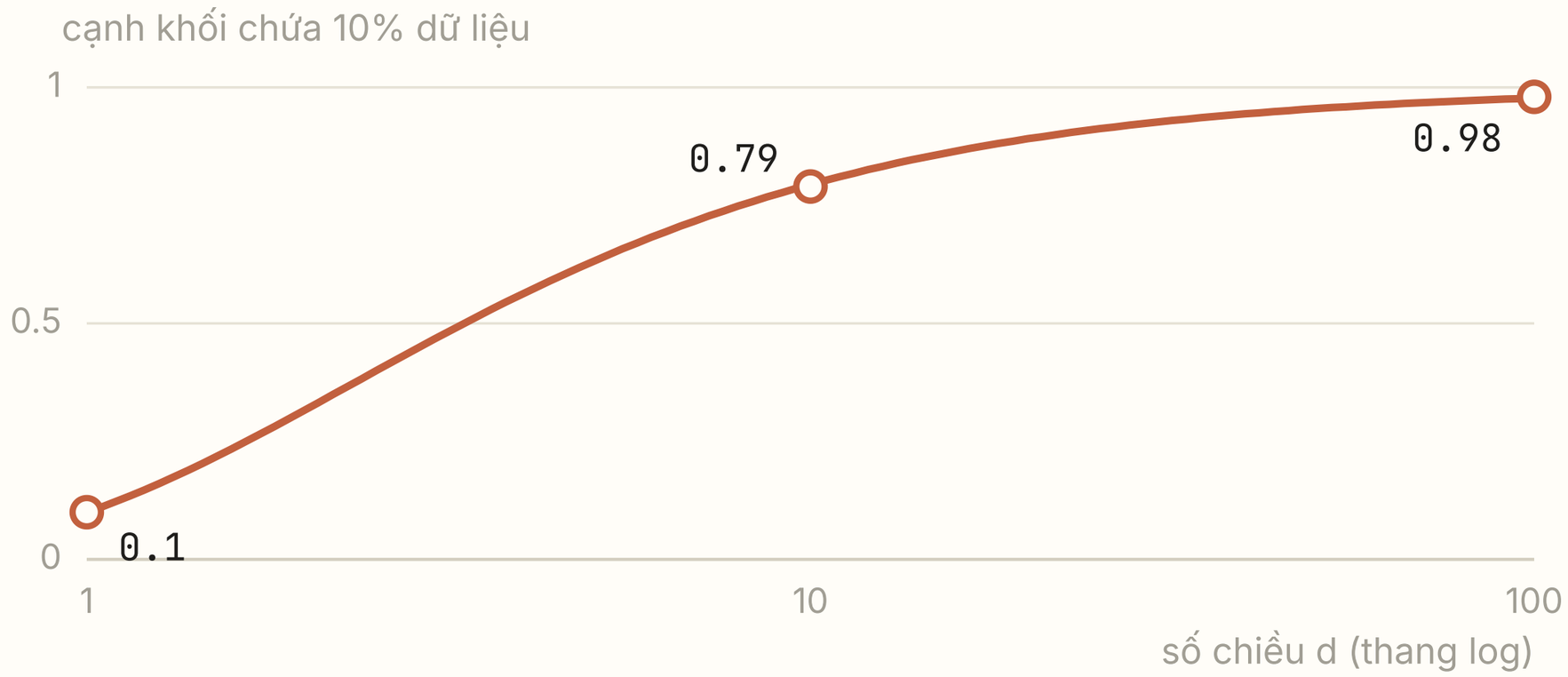
Láng giềng mất ý nghĩa ở nhiều chiều

Với điểm rải đều trong khối lập phương đơn vị, cạnh của vùng chứa 10% dữ liệu là $0.1^{1/d}$:

$$d = 1 : 0.1, \qquad d = 10 : 0.79, \qquad d = 100 : 0.98.$$

Trong 100 chiều, một vùng “lân cận” đã gần bằng cả miền. Chọn đặc trưng hoặc dùng embedding tốt trước khi tăng k.

Cạnh của vùng lân cận



Muốn gom 10% dữ liệu rải đều, cạnh của khối lân cận dài 0.1 ở 1 chiều nhưng 0.98 ở 100 chiều.

HỎI NHANH

Dữ liệu rải đều trong một khối lập phương đơn vị 50 chiều. Một khối lập phương nhỏ chứa 1% dữ liệu thì có cạnh dài khoảng bao nhiêu?

A 0.02

B 0.2

C 0.91

D 0.99

Chi phí và bộ nhớ

Một truy vấn cần tính khoảng cách trong $O(nd)$. Sort toàn bộ tốn $O(n \log n)$; `argpartition` có thể chọn trước k phần tử rồi sort phần nhỏ.

100 000 truy vấn và 100 000 điểm tham chiếu tạo 10^{10} phần tử. Với `float64`, ma trận khoảng cách cần khoảng **80 GB**.

Chia truy vấn thành batch trước khi dùng GPU. KD-tree/Ball tree hợp hơn ở số chiều thấp; ANN đổi một phần độ chính xác lấy tốc độ.

HỎI NHANH

Cần tìm ảnh giống nhau trong 1 triệu ảnh, mỗi ảnh là một embedding 512 chiều, và mỗi truy vấn phải trả lời trong vài mili giây. Cách nào hợp lý?

A KD-tree, vì nó luôn bảo đảm $O(\log n)$ mỗi truy vấn

B Giảm k xuống 1 để mỗi truy vấn tính ít khoảng cách hơn

C Duyệt hết bằng vòng `for` Python qua từng ảnh

D Tìm gần đúng (HNSW, FAISS), chấp nhận đôi khi sót

02

Tự cài và dùng thư viện

Đối chiếu trên máy

Năm điểm A–E và truy vấn $q = (2, 2)$ của ví dụ tính tay.

```
distance2 = np.sum(
    (points - query) ** 2, axis=1
)
order = np.argsort(distance2, kind="stable")
neighbors = order[:3]
counts = np.bincount(
    labels[neighbors], minlength=2
)
print(distance2.astype(int).tolist())
print(neighbors.tolist(), counts.tolist(),
      int(np.argmax(counts)))
```

KẾT QUẢ

```
[2, 1, 5, 13, 1]
[1, 4, 0] [2, 1] 0
```

Chỉ số `[1, 4, 0]` là B, E, A; nhãn `0` thắng hai phiếu. `argmax` cũng là quy tắc phá hòa của đoạn code này.

Chọn k và scaler trong cùng CV

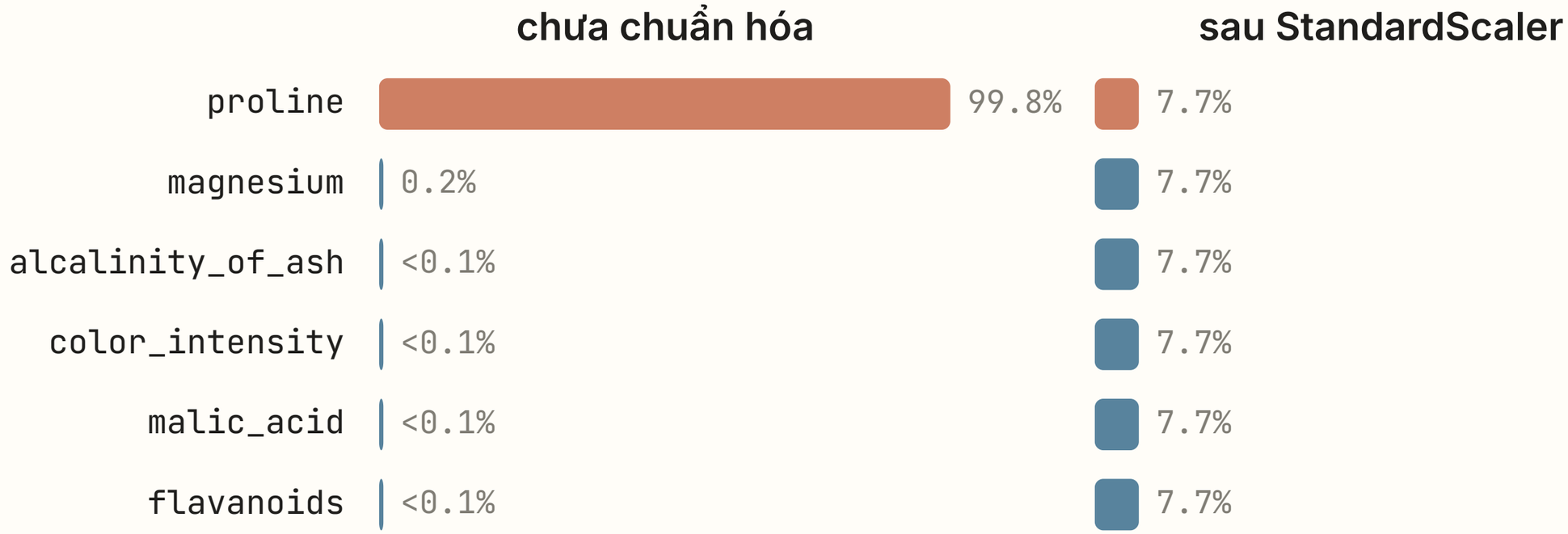
```
model = make_pipeline(StandardScaler(), KNeighborsClassifier())
grid = GridSearchCV(
    model, {"kneighborsclassifier__n_neighbors": [1, 3, 5, 9, 15]}, cv=5
)
grid.fit(X, y)
```

KẾT QUẢ

```
không chuẩn hóa 0.725 {'kneighborsclassifier__n_neighbors': 1}
chuẩn hóa 0.966 {'kneighborsclassifier__n_neighbors': 9}
```

Proline (278–1680) chiếm khoảng 99.8% bình phương khoảng cách khi chưa chuẩn hóa. Scaler fit trong train của từng fold, không fit một lần trên toàn bộ dữ liệu trước CV.

Proline át mọi cột



phần của mỗi cột trong bình phương khoảng cách (6 cột lớn nhất trên 13)

Bộ wine: chưa chuẩn hóa, proline chiếm 99.8% bình phương khoảng cách; sau StandardScaler mỗi cột góp như nhau.

HỎI NHANH

Trong bộ wine chưa chuẩn hóa, riêng cột proline chiếm khoảng 99.8% bình phương khoảng cách. Thêm một feature hữu ích nằm trong khoảng [0, 1], thì kNN với khoảng cách thô sẽ ra sao?

A Gần như không đổi, vì feature mới gần như vô hình

B Tốt hơn hẳn, vì có thêm thông tin

C Tệ hơn hẳn, vì thêm một chiều nữa

D Không chạy được, vì các cột khác thang đo

HỎI NHANH

$k = 3$, ba láng giềng gần nhất mang ba nhãn khác nhau A, B, C, bỏ phiếu đều. Trước khi áp quy tắc xử lý hòa của thư viện, kết quả là gì?

A A thắng, vì là láng giềng gần nhất

B C thắng, vì đứng cuối danh sách

C Không thể hòa, vì k là số lẻ

D Hòa 1–1–1, k lẻ cũng không tránh được

Bẫy cần nhớ

Một k lẻ chỉ tránh hòa phiếu khi có đúng hai lớp và phiếu đồng trọng số. Với ba lớp, $k = 3$ vẫn có thể là 1–1–1. Với trọng số $1/d$, khoảng cách bằng 0 cần quy ước riêng để không chia cho 0.

Khi đi thi

- Có embedding: chuẩn hóa độ dài, kNN cosine làm baseline.
- Đọc quy tắc phá hòa và kiểu khoảng cách trong đề.
- Scaler, chọn feature, giảm chiều đặt trong Pipeline trước khi dò k.
- Nhiều truy vấn: đo RAM, chia batch.

03

Luyện đề

LUYỆN ĐỀ 1

Câu nào về kNN **luôn** đúng?

A Giảm k làm ranh giới quyết định đơn giản hơn

B kNN chỉ dùng cho phân loại, không dùng cho hồi quy

C Tăng k làm mô hình ít nhạy với điểm ngoại lai

D kNN chỉ dùng được với feature là số thực

PHỎNG THEO CMU 10-601 · Exam 1 Practice · Q2.4

LUYỆN ĐỀ 2

Dữ liệu một chiều, n điểm đã sắp xếp, phân loại hai lớp bằng 1-NN với khoảng cách Euclid. Có dựng được một quy tắc chỉ gồm các phép so sánh dạng $x \geq t$, nối tiếp nhau, mà cho dự đoán giống hệt 1-NN trên mọi đầu vào không?

A Được, đặt ngưỡng ở trung điểm của hai điểm kề nhau khác nhãn

B Không được, vì so sánh từng ngưỡng không bắt chước được khoảng cách

C Chỉ được khi các điểm train cách đều nhau trên trục

D Chỉ được nếu dùng k lẻ thay vì $k = 1$

PHỎNG THEO CMU 10-601 · Exam 1 Practice · Q2.5

LUYỆN ĐỀ 3

Đề phân loại hai lớp, 30 feature số, chấm bằng macro-F1. Đề **bắt buộc** kNN là mô hình học có giám sát duy nhất, còn tiền xử lý thì tùy ý. Hướng làm nào hợp lý nhất?

A Chỉ dò thật kỹ k, vì k là tham số duy nhất của kNN

B Học một mô hình khác như boosting trước, rồi lấy dự đoán của nó làm feature cho kNN

C Chuẩn hóa, chọn feature trong Pipeline, rồi dùng cross-validation theo macro-F1

D Fit `StandardScaler` trên cả train lẫn test cho cùng thang

PHỎNG THEO USA-NA-AIO 2026 · Vòng 1 · Problem 9

Chốt ba điều

kNN chỉ lưu dữ liệu rồi dùng láng giềng để dự đoán.

Metric và thang đo quyết định ai là láng giềng; k quyết định độ mượt.

CV phải chọn cả preprocessing và k trên cùng giao thức.

Sau giờ học

Làm **10 câu kiểm tra cuối bài** trên trang bài học.

Chạy notebook từ đầu; đổi đơn vị một cột, dự đoán trước việc xếp hàng xóm, rồi kiểm lại bằng output.